

StaticKD: Klasifikasi Topik Berita IPTC Multibahasa yang Ringan untuk CPU melalui Distilasi Pengetahuan ke Model *Static-Embedding*

Obi Kastanya and Ananta Dwi Prayoga Alwy

Abstrak—Klasifikasi artikel berita ke dalam taksonomi IPTC Media Topic dibutuhkan untuk menyaring jutaan artikel per hari, tetapi pengklasifikasi multibahasa terbaik yang tersedia secara terbuka, berbasis XLM-RoBERTa-large, membutuhkan 0,9–2,6 detik per artikel pada satu *core* CPU. Penelitian ini mengusulkan StaticKD, yaitu distilasi pengetahuan dari pengklasifikasi tersebut ke model *student* berbasis *static embedding* multibahasa: embedding token pra-latih yang dirata-rata lalu dipetakan secara linear ke 17 kelas. *Teacher* melabeli 286 ribu artikel dari 70 bahasa, dan *student* dilatih meminimalkan divergensi Kullback–Leibler terhadap distribusi *teacher* yang dilunakkan temperatur. Karena kepala klasifikasinya linear, model akhir dapat dilipat menjadi tabel berisi 17 logit per token (34 MB) yang dijalankan hanya dengan NumPy. Pada data dev EMMediaTopic (4 bahasa) dan *test set* 16 bahasa di luar data latih *teacher*, StaticKD mencapai macro-F1 0,751 dan 0,740 (sekitar 90% dari *teacher*) dengan *throughput* 447 artikel per detik per *core*, sekitar 720 kali lebih cepat dari *teacher* ONNX int8. StaticKD tidak berbeda signifikan dari *student* MiniLM-L6 yang 36 kali lebih lambat, dan mengungguli fastText hasil distilasi yang sama pada bahasa di luar data latih (+0,084 macro-F1). Ablasi menunjukkan bahwa kunci keberhasilannya adalah data distilasi multibahasa dan embedding yang selaras lintas bahasa. Penambahan kapasitas yang tetap dapat dilipat tidak membantu, sedangkan penurunan temperatur ke $\tau = 1$ disertai *ensemble* tabel memberi kenaikan kecil (macro-F1 dev 0,761). Kelemahan per bahasa terutama ditentukan oleh jumlah data distilasi dan keyakinan *teacher*, bukan oleh sistem tulisan.

Kata Kunci—Klasifikasi topik berita, IPTC Media Topic, *knowledge distillation*, *static embedding*, klasifikasi teks multibahasa, efisiensi inferensi, CPU.

I. PENDAHULUAN

ARTIKEL berita adalah salah satu sumber informasi utama dalam industri data analitik karena sifatnya yang faktual dan terkini. Agar analisis tetap terarah, artikel perlu dikelompokkan ke kategori terstandar, lalu hanya topik yang relevan yang diproses lebih lanjut. International Press Telecommunications Council (IPTC) Media Topic adalah standar pengkategorian berita yang dipakai kantor berita besar seperti Reuters, Associated Press, dan Agence France-Presse [1], [2]. Level teratasnya terdiri dari 17 topik, antara lain *politics*, *economy*, *business and finance*, dan *sport*.

Kuzman dan Ljubešić [1] menunjukkan bahwa *large language model* (LLM) dapat menggantikan anotator manusia

dalam membangun data latih klasifikasi IPTC. GPT-4o melabeli 20.000 artikel berita berbahasa Slovenia, Kroasia, Katalan, dan Yunani secara *zero-shot*, dengan kesepakatan terhadap manusia yang setara dengan kesepakatan antarmanusia. Label tersebut dipakai untuk melakukan *fine-tuning* XLM-RoBERTa-large [3], dan model yang dihasilkan (macro-F1 0,746 pada *test set* manusia) dirilis sebagai pengklasifikasi IPTC multibahasa terbuka pertama. Namun penelitian tersebut tidak mengukur dimensi efisiensi: jumlah parameter, ukuran model, memori, dan latensi inferensi. Padahal alasan utama mengganti GPT dengan model yang lebih kecil adalah kebutuhan memproses jutaan artikel per hari. Model berbasis XLM-RoBERTa-large dengan 560 juta parameter pada praktiknya menuntut GPU, dan kebutuhan itu tidak terpenuhi oleh banyak *pipeline* produksi yang berjalan di server kecil berbasis CPU.

Kebutuhan praktis yang kami targetkan serupa dengan fastText *language identification* [4]: satu model kecil yang mengenali banyak bahasa dalam hitungan milidetik di CPU. Proposal awal penelitian ini mempertimbangkan kombinasi *knowledge distillation* (KD) dan *low-rank adaptation* (LoRA) [5], [6]. Akan tetapi, LoRA hanya mengurangi jumlah parameter yang dilatih, bukan biaya *inferensi*: setelah adapter digabungkan, model tetap sebesar *backbone*-nya. Karena itu penelitian ini beralih ke pertanyaan yang lebih mendasar: seberapa sederhana arsitektur *student* yang masih mampu meniru *teacher* IPTC multibahasa?

Pertanyaan penelitian (*research question*, RQ) yang dijawab adalah sebagai berikut.

- **RQ1:** Seberapa akurat dan seberapa mahal pengklasifikasi IPTC Kuzman dan Ljubešić bila dijalankan hanya dengan CPU?
- **RQ2:** Dapatkah pengetahuan *teacher* didistilasi ke model *static-embedding* secepat fastText tanpa kehilangan kemampuan multibahasa?
- **RQ3:** Komponen apa yang paling menentukan keberhasilan distilasi: sumber label, cakupan bahasa data, inisialisasi embedding, regularisasi, ukuran data, atau temperatur?
- **RQ4:** Bagaimana *trade-off* akurasi–efisiensi dibanding fastText dan *student* transformer kecil, dan apakah ada bahasa atau label yang sistematis lebih lemah?

Kontribusi penelitian ini adalah sebagai berikut.

- 1) Laporan efisiensi pertama pengklasifikasi IPTC Kuzman dan Ljubešić di CPU (latensi, *throughput*, memori, ukuran) dalam dua konfigurasi (PyTorch fp32 dan ONNX

O. Kastanya (NRP 6025252015) dan A. D. P. Alwy (NRP 6025252007) berasal dari [TODO: program studi, departemen, institusi, kota, negara]. E-mail: [TODO: alamat e-mail penulis korespondensi].

Naskah diterima [TODO: tanggal]; direvisi [TODO: tanggal].

int8), dengan protokol *benchmark* yang dapat direproduksi untuk CPU *hybrid*.

- 2) StaticKD, resep distilasi *teacher* ke *static multilingual embedding* untuk klasifikasi topik berita yang menghasilkan model 34 MB dengan *throughput* ratusan artikel per detik per *core*, beserta ablasi kontribusi setiap komponen dan eksplorasi batas akurasi.
- 3) Evaluasi lintas bahasa pada 16 bahasa di luar data latih *teacher* dan 70 bahasa *held-out*, termasuk analisis statistik faktor penentu kelemahan per bahasa.
- 4) Analisis kalibrasi dan mode *cascade student-teacher* untuk kasus yang membutuhkan akurasi setara *teacher*.

Bagian II membahas penelitian terkait. Bagian III menjelaskan data, metode, dan protokol evaluasi. Bagian IV menyajikan hasil, Bagian V membahasnya, Bagian VI memuat keterbatasan, dan Bagian VII menyimpulkan.

II. PENELITIAN TERKAIT

A. Klasifikasi Topik Berita Berbasis IPTC

Dataset topik berita sebelumnya memakai skema sendiri, label yang tidak dianotasi manual, atau skema IPTC Subject Codes yang sudah usang [1]. Kuzman dan Ljubešić [1] membangun EMMediaTopic, 21.000 artikel dari korpus MaCoCu (diekstrak dengan pengklasifikasi genre X-GENRE [7]) yang dilabeli GPT-4o dengan 17 topik level atas IPTC Media Topic, beserta *test set* manusia berisi 1.129 artikel. XLM-RoBERTa-large yang dilatih pada 15.000 artikel mencapai macro-F1 0,746, setara GPT-4o (0,731), dan model multibahasa tidak kalah dari model monolingual. *Benchmark* lanjutan mereka [8] menunjukkan bahwa pengklasifikasi ini bersaing dengan GPT-4o, GPT-5, dan Gemini 2.5, sedangkan *baseline* non-neural (TF-IDF + SVC) hanya mencapai macro-F1 0,42. Tidak satu pun studi tersebut melaporkan biaya inferensi.

B. Parameter-Efficient Fine-Tuning

Razuvayevskaya dkk. [9] membandingkan LoRA, adapter, dan *full fine-tuning* pada XLM-RoBERTa-large untuk klasifikasi berita multibahasa. PEFT memangkas parameter terlatih 140–280 kali dengan akurasi serupa. Azimi dkk. [6] menggabungkan KD dan LoRA (KD-LoRA) dan memperoleh 97% kinerja *full fine-tuning* dengan penurunan latensi 30% pada GLUE. Penurunan latensi sebesar itu tidak cukup untuk kasus kami, yang membutuhkan penurunan dua sampai tiga orde besaran.

C. Knowledge Distillation dan Model Ringan

Di antara teknik kompresi model (pemangkasan, kuantisasi, distilasi, faktorisasi) [10], KD [11] melatih *student* untuk meniru distribusi probabilitas *teacher* yang dilunakkan dengan temperatur. Distribusi ini membawa informasi kemiripan antarkelas yang tidak ada pada label keras. Efektivitas KD bergantung pada kesenjangan kapasitas *teacher* dan *student* [12], [13], dan kesesuaian prediksi *student* dengan *teacher* (*fidelity*) tidak selalu sejalan dengan generalisasinya [14]. Varian transformer kecil seperti DistilBERT [15] dan MiniLMv2 [16] mengurangi jumlah *layer*, tetapi tetap

memerlukan *attention* atas seluruh token. Di sisi lain spektrum, fastText [4] adalah model *bag-of-n-gram* yang sangat cepat, tetapi embedding-nya dilatih dari nol sehingga tidak ada transfer lintas bahasa. Model2Vec [17] mendistilasi *sentence transformer* menjadi embedding token statis. Varian *potion-multilingual-128M* didistilasi dari BGE-M3 [18] untuk 101 bahasa, sehingga kata yang bermakna sama di bahasa berbeda memiliki vektor yang berdekatan. Piperno dkk. [19] menunjukkan bahwa distilasi efektif memindahkan pengetahuan antarbahasa, terutama ketika data bahasa target sedikit.

D. Kesenjangan Penelitian

Ada tiga kesenjangan. Pertama, pengklasifikasi IPTC yang ada tidak dievaluasi efisiensinya di CPU. Kedua, distilasi *teacher* transformer ke *static embedding* multibahasa belum diuji untuk klasifikasi topik berita dengan banyak kelas. Ketiga, belum ada analisis sistematis mengenai bahasa mana yang lemah setelah model dikompresi dan mengapa. Ulčar dkk. [20] menunjukkan bahwa penurunan kinerja lintas bahasa bervariasi antartugas sehingga harus diukur langsung.

III. METODOLOGI

A. Kerangka Umum

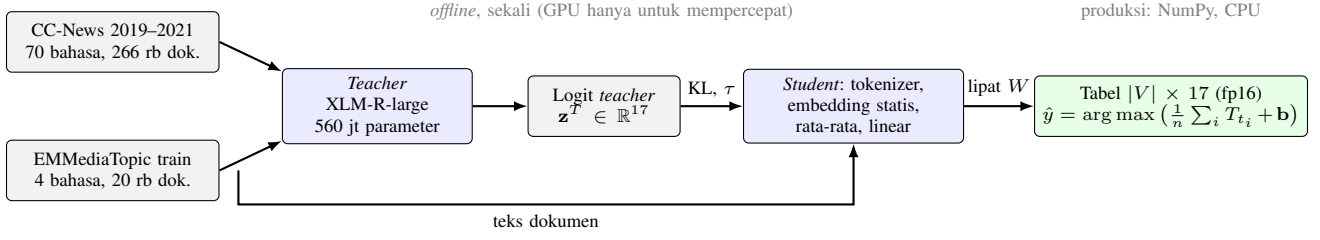
Gbr. 1 merangkum *pipeline* StaticKD. Pelabelan oleh *teacher* dilakukan sekali secara *offline* (GPU hanya dipakai untuk mempercepat proses ini), sedangkan model produksi berjalan sepenuhnya di CPU.

B. Data

EMMediaTopic 1.0 [1] berisi 21.000 artikel (hr, sl, ca, el; 5.250 per bahasa) berlabel GPT-4o. Split *train* (20.000) dipakai untuk distilasi. Split *dev* (1.000) tidak pernah dipakai untuk melatih *teacher* maupun *student* dan menjadi set evaluasi utama.

Korpus CC-News multibahasa diambil dari arsip Common Crawl News 2019–2021 yang dipisah per bahasa [21], dengan aturan berikut: (i) hanya dokumen yang bahasanya sama menurut metadata dan fastText-LID; (ii) judul dan isi, minimal 75 kata (200 karakter untuk ja, zh, th, my), dipotong ke 512 kata mengikuti praproses [1]; (iii) maksimal 150 dokumen per situs; (iv) pemisahan *held-out* per domain, yaitu seluruh dokumen dari sebagian situs disisihkan (maksimal 20% per bahasa) sehingga evaluasi *held-out* mengukur generalisasi ke situs berita yang belum pernah dilihat. Hasilnya adalah 109.842 dokumen latih (69 bahasa) dan 13.549 dokumen *held-out* (68 bahasa). Untuk eksperimen skala data ditambahkan 156.157 dokumen (maksimal 5.000 per bahasa, tidak pernah dari domain *held-out*), sehingga total data distilasi menjadi 285.999 dokumen.

Test set LLM 16 bahasa (selanjutnya LLM16) terdiri dari 320 artikel *held-out* (20 per bahasa: en, de, fr, es, pt, it, ru, pl, tr, ar, zh, ja, id, vi, hi, sw) yang disebar merata antarsitus. Semua bahasa ini tidak ada di data *fine-tuning teacher*. Artikel dianotasi oleh LLM (Claude) menggunakan *prompt* dan deskripsi label yang persis sama dengan yang dipakai Kuzman dan Ljubešić untuk GPT-4o [1, Lampiran B],



Gbr. 1. Pipeline StaticKD. *Teacher* melabeli korpus berita tak berlabel sekali secara *offline*; *student* dilatih meniru distribusi *teacher*, lalu kepala linearnya dilipat ke dalam tabel token sehingga inferensi hanya berupa pencarian baris tabel dan rata-rata.

Tabel I
RINGKASAN DATA

Korpus	Dokumen	Bahasa	Label	Peran
EMMediaTopic train	20.000	4	GPT-4o + logit <i>teacher</i>	distilasi
EMMediaTopic dev	1.000	4	GPT-4o	evaluasi utama
CC-News train	109.842	69	logit <i>teacher</i>	distilasi
CC-News tambahan	156.157	50	logit <i>teacher</i>	skala data
CC-News <i>held-out</i>	13.549	68	logit <i>teacher</i>	<i>fidelity</i>
LLM16	320	16	LLM (<i>prompt</i> [1])	lintas bahasa

tanpa akses ke prediksi model mana pun. Ringkasan data ditunjukkan pada Tabel I.

Test set manusia. [TODO: Bagian ini akan diisi setelah *test set* manusia selesai: sekitar 1.800 artikel CC-NEWS 2024–2026 dari situs yang tidak ada di data latih, 6 bahasa, sampling acak ditambah stratifikasi rubrik URL, deduplikasi *near-duplicate* terhadap data latih, anotasi ganda, dan Cohen’s κ .]

C. Teacher dan Pelabelan Distilasi

Teacher adalah classla/multilingual-IPTC-news-topic-classifier (XLM-RoBERTa-large, 559,9 juta parameter, maksimal 512 token). Untuk setiap dokumen d di korpus distilasi, *teacher* menghasilkan logit $\mathbf{z}^T(d) \in \mathbb{R}^{17}$ yang disimpan utuh. Pelabelan dijalankan dalam fp16 dengan *batch* yang diurutkan berdasarkan panjang teks, dengan kecepatan sekitar 51 dokumen per detik pada GPU RTX 4050. Distribusi label *teacher* pada CC-News masuk akal (*politics* 15,6%, *economy* 13,0%, *arts* 11,3%, *sport* 10,5%, *weather* 1,2%), dan keyakinan rata-rata *teacher* per bahasa berkisar 0,88–0,98.

D. Student StaticKD

Arsitektur. Dokumen d ditokenisasi menjadi $t_1, \dots, t_n$ menggunakan tokenizer Unigram 500.353 token milik *potion-multilingual-128M* [17] (maksimal 1.024 token). Setiap token memiliki embedding $\mathbf{e}_t \in \mathbb{R}^{256}$, dan

$$\mathbf{h}(d) = \frac{1}{n} \sum_{i=1}^n \mathbf{e}_{t_i}, \quad \mathbf{z}^S(d) = W\mathbf{h}(d) + \mathbf{b}, \quad (1)$$

dengan $W \in \mathbb{R}^{17 \times 256}$. Matriks embedding E diinisialisasi dari *potion-multilingual-128M*, yaitu embedding statis 101 bahasa yang selaras lintas bahasa.

Fungsi loss. Dengan $p^T = \text{softmax}(\mathbf{z}^T/\tau)$ dan $p^S = \text{softmax}(\mathbf{z}^S/\tau)$,

$$\mathcal{L}_{\text{KD}} = \tau^2 \text{KL}(p^T \| p^S). \quad (2)$$

Model utama memakai $\tau = 2$ (varian $\tau = 1$ dibahas pada Subbagian IV-D). Parameter E , W , dan $\mathbf{b}$ dilatih *end-to-end*. E memakai SparseAdam (*learning rate* 10^{-3}) sehingga hanya baris token yang muncul di *batch* yang diperbarui, sedangkan W dan $\mathbf{b}$ memakai AdamW (*learning rate* $3 \cdot 10^{-3}$). Ukuran *batch* 64 dengan 6 *epoch*.

Token dropout. Setiap token dokumen latih dibuang dengan probabilitas 0,2 sebagai augmentasi agar model tidak bergantung pada beberapa kata kunci.

Pemilihan model. Sebanyak 3% data latih disisihkan sebagai validasi internal dengan metrik kesepakatan terhadap *teacher*. Set evaluasi tidak pernah dipakai untuk memilih hiperparameter.

Pelipatan tabel. Karena kepala klasifikasinya linear,

$$W \left(\frac{1}{n} \sum_i \mathbf{e}_{t_i} \right) = \frac{1}{n} \sum_i T_{t_i}, \quad T = EW^T \in \mathbb{R}^{|V| \times 17}. \quad (3)$$

Model produksi hanyalah tabel T dalam fp16 ditambah bias dan tokenizer. Inferensi terdiri dari tokenisasi, rata-rata baris tabel, dan penambahan bias, tanpa perkalian matriks dan tanpa PyTorch.

Varian compact. Struktur internal tokenizer Unigram 500 ribu token membutuhkan sekitar 490 MB RAM. Varian *compact* membuang potongan token yang muncul kurang dari 10 kali di korpus distilasi (karakter tunggal tetap dipertahankan), sehingga kosakata menjadi 335.441 token.

E. Baseline

- 1) **fastText** [4] (dimensi 100, bigram kata, *learning rate* 0,5, softmax) dalam beberapa varian: dilatih dengan label GPT-4o EMMediaTopic saja (setting Kuzman, 4 bahasa) atau dengan argmax *teacher* (70 bahasa), masing-masing dengan token kata atau subword SentencePiece XLM-R, ditambah varian terkuantisasi (`.ftz`).
- 2) **MiniLM-L6 + KD**: mMiniLMv2-L6-H384 yang didistilasi dari XLM-R-large [16] dengan *loss* KD yang sama ($\tau = 2$), 2 *epoch*, *learning rate* $5 \cdot 10^{-5}$, maksimal 256 token, lalu diekspor ke ONNX fp32 dan int8 dinamis.
- 3) **Teacher** dalam dua konfigurasi CPU: PyTorch fp32 (sesuai *model card*) dan ONNX int8 (ekspor onnx-community), yang merupakan konfigurasi CPU terbaik untuk *teacher*.

F. Protokol Evaluasi dan Uji Statistik

Metrik utama adalah macro-F1 seperti pada [1], ditambah micro-F1 (sama dengan akurasi). Interval kepercayaan 95% di-

hitung dengan *bootstrap percentile* (1.000 *resampling*). Perbedaan dua model pada dokumen yang sama diuji dengan *paired bootstrap* (2.000 *resampling*) untuk Δ macro-F1 [22] dan uji McNemar eksak untuk akurasi [23], dengan koreksi Holm–Bonferroni [24] per set evaluasi. Validitas LLM16 diukur dengan Cohen’s κ dan Krippendorff’s α [25] antara anotator LLM dan *teacher*. Kalibrasi diukur dengan *expected calibration error* (ECE, 15 bin) [26]. Analisis per bahasa memakai korelasi Spearman, uji Kruskal–Wallis, dan perbandingan R^2 regresi. Pada eksplorasi Subbagian IV-D, varian dibandingkan dengan tiga *seed* dan uji Welch pada tingkat *seed*.

G. Protokol Efisiensi

Benchmark dijalankan hanya di CPU (GPU disumbangkan), dengan satu model per proses baru. Jumlah *thread* dikunci pada seluruh pustaka (OMP, MKL, ONNX Runtime, *tokenizers*). Latensi diukur dengan satu dokumen per panggilan (skenario *streaming*: rata-rata, p50, p95), sedangkan *throughput* diukur dengan *batch* 64 untuk *student* ringan atau 8 untuk *transformer*. RAM model adalah kenaikan RSS setelah model dimuat. Sampel teks adalah dokumen dev (median 220 kata). Perangkat keras yang digunakan adalah Intel Core Ultra 7 155H dengan RAM 31 GB dan Windows 11. CPU ini bersifat *hybrid* (6 P-core, 8 E-core yang sekitar 2,5 kali lebih lambat, dan 2 LP E-core yang sekitar 3,7 kali lebih lambat). Tanpa *pinning*, sistem operasi memindahkan *thread* ke *core* lambat sehingga hasil *multi-thread* tidak stabil. Karena itu setiap proses dipin ke P-core fisik (1 *core*: CPU logis 0; 4 *core*: CPU logis 0, 10, 12, 14) untuk mensimulasikan server kecil dengan *core* seragam.

IV. HASIL

A. Akurasi dan Efisiensi Teacher (RQ1)

Tabel II menunjukkan akurasi dan efisiensi *teacher*. Skor dev lebih tinggi dari 0,746 pada paper asli karena labelnya berasal dari GPT-4o, yaitu label yang memang ditiru *teacher*. *Teacher* sangat kuat lintas bahasa: akurasi per bahasa pada LLM16 berkisar 0,65 (hi, ru) hingga 1,00 (es, sw). Kesepakatan anotator LLM dengan *teacher* adalah 83,1% ($\kappa = 0,815$; $\alpha = 0,816$, CI95 0,764–0,856), setara atau lebih tinggi dari kesepakatan antardua anotator manusia pada paper Kuzman ($\alpha = 0,728$). Hal ini menunjukkan LLM16 layak sebagai rujukan tambahan. Kebingungan utama *teacher*, yaitu *human interest* dengan *arts*, *culture*, *entertainment and media* serta *society* dan *economy* yang tertukar ke *politics*, konsisten dengan [1].

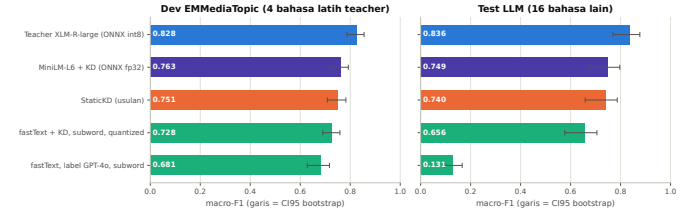
Jawaban RQ1: *teacher* akurat dan kuat lintas bahasa, tetapi untuk satu juta artikel per hari (sekitar 11,6 artikel per detik) dibutuhkan sekitar 18,7 *core* (ONNX int8) sampai 52,6 *core* (PyTorch) khusus untuk klasifikasi. Biaya ini tidak layak untuk *pipeline real-time* di server kecil.

B. Akurasi Seluruh Model (RQ2, RQ4)

Hasil utama ditunjukkan pada Tabel III dan Gbr. 2. StaticKD mempertahankan 90,7% macro-F1 *teacher* ONNX di dev dan 88,5% di 16 bahasa lain. Kuantisasi int8 dinamis tidak

Tabel II
AKURASI DAN EFISIENSI *Teacher* DI CPU (P-core TERPIN)

	PyTorch fp32	ONNX int8
Macro-F1 dev [CI95]	0,820 [0,779–0,849]	0,828 [0,787–0,855]
Micro-F1 dev	0,858	0,860
Macro-F1 LLM16 [CI95]	0,830 [0,769–0,872]	0,836 [0,770–0,877]
Micro-F1 LLM16	0,831	0,838
Latensi p50 / p95, 1 <i>core</i> (ms)	2.600 / 4.415	913 / 1.942
Latensi p50 / p95, 4 <i>core</i> (ms)	1.083 / 1.710	329 / 527
Throughput 1 / 4 <i>core</i> (dok/s)	0,22 / 0,59	0,62 / 2,08
RAM model (MB)	2.957	1.383
Ukuran file (MB)	2.136	537



Gbr. 2. Macro-F1 dengan interval kepercayaan 95% *bootstrap* pada dua set evaluasi berlabel.

merugikan *teacher*, tetapi menurunkan MiniLM-L6 secara drastis (−0,130 macro-F1 dev), bahkan dengan kuantisasi *per-channel* yang hanya diterapkan pada operasi MatMul. Karena itu MiniLM dilaporkan dalam fp32.

Uji berpasangan pada Tabel IV menghasilkan tiga temuan. Pertama, StaticKD lebih rendah secara signifikan dari *teacher*. Kedua, StaticKD tidak berbeda signifikan dari MiniLM-L6 + KD di kedua set. Ketiga, StaticKD signifikan lebih baik dari fastText + KD pada 16 bahasa lain, sedangkan pada 4 bahasa latih perbedaannya tidak signifikan. Jadi keunggulan StaticKD terutama terletak pada generalisasi lintas bahasa.

C. Ablasi (RQ3)

Tabel V dan Gbr. 3 menunjukkan kontribusi setiap komponen. Setiap baris mengubah satu faktor.

Pembacaan ablasi adalah sebagai berikut.

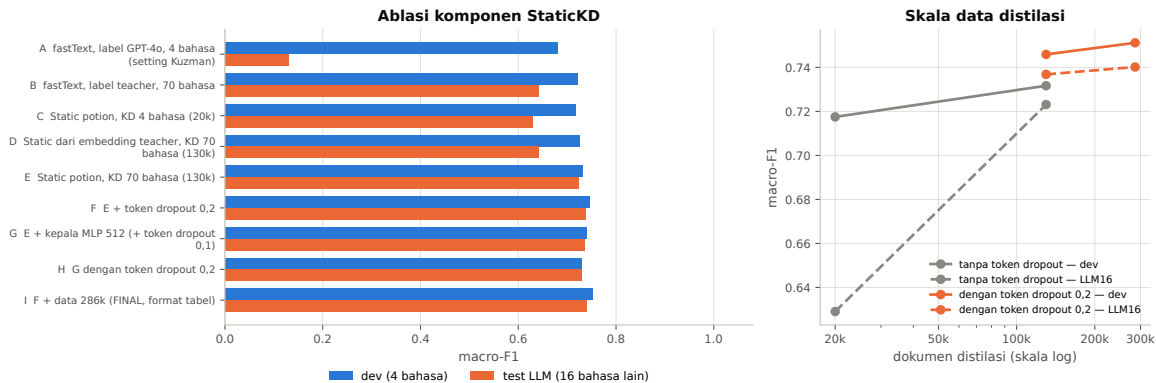
- **Distilasi multibahasa adalah komponen terpenting.** Dari A ke B, macro-F1 lintas bahasa naik 0,511. Tanpa data dari bahasa lain, model *bag-of-words* gagal total di luar 4 bahasa latih.
- **Embedding yang selaras lintas bahasa memberi transfer tanpa data bahasa target.** Dari A ke C, macro-F1 LLM16 naik dari 0,131 ke 0,629, dan gabungan keduanya adalah yang terbaik (E).
- **Embedding yang didistilasi dari encoder teacher** (D) kalah dari potion untuk lintas bahasa (0,642 vs 0,723). Representasi token tunggal dari *encoder* yang di-*fine-tune* ternyata kurang selaras antarbahasa.
- **Token dropout** memberi +0,014 (F), sedangkan kepala MLP tidak memberi tambahan (G, H) dan tidak dapat dilipat, sehingga ditolak.
- **Pelipatan tabel tidak mengubah prediksi** (J = F), sementara ukuran model turun dari 128 MB (embedding int8) menjadi 34 MB.

Tabel III
HASIL UTAMA. “KESEPAKATAN” ADALAH AKURASI TERHADAP PREDIKSI *Teacher* PADA 13.549 DOKUMEN *Held-Out* DARI 70 BAHASA

Model	Macro-F1 dev [CI95]	Micro-F1 dev	Macro-F1 LLM16 [CI95]	Micro-F1 LLM16	Kesepakatan
<i>Teacher</i> (PyTorch fp32)	0,820 [0,779–0,849]	0,858	0,830 [0,769–0,872]	0,831	–
<i>Teacher</i> (ONNX int8)	0,828 [0,787–0,855]	0,860	0,836 [0,770–0,877]	0,838	–
MiniLM-L6 + KD (ONNX fp32)	0,763 [0,718–0,792]	0,801	0,749 [0,678–0,797]	0,766	0,814
MiniLM-L6 + KD (ONNX int8)	0,633 [0,575–0,675]	0,725	0,610 [0,523–0,668]	0,669	–
StaticKD (usulan)	0,751 [0,708–0,783]	0,793	0,740 [0,659–0,787]	0,744	0,800
StaticKD-compact	0,751 [0,708–0,783]	0,793	0,730 [0,647–0,776]	0,738	0,789
fastText + KD, subword, terkuantisasi	0,728 [0,690–0,759]	0,764	0,656 [0,577–0,705]	0,669	0,698
fastText + KD, subword	0,721 [0,677–0,754]	0,767	0,642 [0,567–0,692]	0,675	0,701
fastText + KD, kata	0,717 [0,675–0,750]	0,764	0,619 [0,540–0,668]	0,666	0,711
fastText, label GPT-4o, subword	0,681 [0,628–0,717]	0,734	0,131 [0,096–0,166]	0,225	–
fastText, label GPT-4o, kata	0,652 [0,603–0,692]	0,738	0,118 [0,076–0,155]	0,222	–

Tabel IV
UJI BERPASANGAN STATICKD TERHADAP MODEL LAIN (*p* DIKOREKSI HOLM; *b/c* = JUMLAH DOKUMEN YANG HANYA BENAR DI STATICKD / HANYA BENAR DI MODEL PEMBANDING)

Set	StaticKD vs	Δ macro-F1 [CI95]	<i>p bootstrap</i>	Δ akurasi	McNemar <i>b/c</i>	<i>p</i> McNemar
dev	<i>Teacher</i> ONNX int8	−0,076 [−0,109; −0,042]	<0,001	−0,067	50/117	<0,001
	<i>Teacher</i> PyTorch	−0,069 [−0,103; −0,034]	<0,001	−0,065	47/112	<0,001
	MiniLM-L6 + KD	−0,012 [−0,046; +0,026]	1,000	−0,008	79/87	0,743
	fastText + KD (q)	+0,023 [−0,018; +0,063]	0,786	+0,029	83/54	0,049
	fastText, GPT-4o	+0,070 [+0,029; +0,119]	<0,001	+0,059	122/63	<0,001
	StaticKD 130 rb	+0,005 [−0,013; +0,023]	1,000	+0,007	26/19	0,743
LLM16	<i>Teacher</i> ONNX int8	−0,096 [−0,171; −0,020]	0,044	−0,094	11/41	<0,001
	<i>Teacher</i> PyTorch	−0,090 [−0,170; −0,017]	0,044	−0,088	14/42	0,001
	MiniLM-L6 + KD	−0,008 [−0,079; +0,057]	1,000	−0,022	30/37	0,928
	fastText + KD (q)	+0,084 [+0,022; +0,148]	0,040	+0,075	41/17	0,007
	fastText, GPT-4o	+0,610 [+0,534; +0,662]	<0,001	+0,519	172/6	<0,001
	StaticKD 130 rb	+0,003 [−0,032; +0,037]	1,000	+0,003	10/9	1,000



Gbr. 3. Ablasi komponen (kiri) dan skala data distilasi (kanan).

- **Menggandakan data hanya memberi +0,005** (I; tidak signifikan, Tabel IV), yang menandakan kapasitas model *bag-of-tokens* sudah jenuh.

D. Eksplorasi Peningkatan Akurasi dan StaticKD v2

Untuk menguji apakah batas akurasi tersebut dapat dilampaui tanpa mengorbankan sifat ringannya, kami menguji perubahan yang mempertahankan format tabel token dan biaya inferensi $O(n)$. Konfigurasi dipilih berdasarkan dev, kesepakatan 70 bahasa dipakai sebagai sinyal pendukung, dan LLM16 sebagai konfirmasi. Hasilnya ditunjukkan pada Tabel VI.

Empat temuan diperoleh dari eksplorasi ini.

- **Variasi antar-*seed* pada LLM16 besar.** Tiga *seed* (M) memberi simpangan baku 0,002 di dev tetapi 0,016 di LLM16 (320 dokumen). Selisih di bawah sekitar 0,03 pada LLM16 tidak dapat diinterpretasikan tanpa pengu-langan *seed*.
- **Fidelity tidak sama dengan akurasi.** Bigram *hashing* (O, P), yaitu tabel 17 logit per pasangan token berurutan yang dilatih dari nol, menaikkan kesepakatan dengan *teacher* secara signifikan tetapi kecil (+0,004; *paired bootstrap* $p = 0,02$), sedangkan akurasinya terhadap label independen turun (dev −0,013, $p = 0,058$; LLM16 −0,018, tidak signifikan) dan ukuran file naik 2–3 kali.

Tabel V
ABLASI KOMPONEN STATICKD

Kode	Varian	dev	LLM16	Kesep.
A	fastText, label GPT-4o, 4 bahasa	0,681	0,131	—
B	fastText, label <i>teacher</i> , 70 bahasa	0,721	0,642	0,701
C	Static potion, KD 4 bahasa (20 rb)	0,718	0,629	—
D	Static dari embedding <i>teacher</i> , KD 130 rb	0,726	0,642	0,748
E	Static potion, KD 70 bahasa (130 rb)	0,732	0,723	0,791
F	E + <i>token dropout</i> 0,2	0,746	0,737	0,796
G	E + kepala MLP 512 (<i>dropout</i> 0,1)	0,739	0,736	0,801
H	G dengan <i>token dropout</i> 0,2	0,729	0,729	0,801
J	F dilipat ke tabel fp16	0,746	0,737	0,796
I	F + data 286 rb (final)	0,751	0,740	0,800
K	I + tokenizer dipangkas (min. 10)	0,751	0,730	0,789
L	I + tokenizer dipangkas (min. 50)	0,749	0,714	0,773

Tabel VI
EKSPLORASI PENINGKATAN AKURASI (RESEP I SEBAGAI TITIK AWAL)

Kode	Varian	dev	LLM16	Kesep.
M	I, 3 <i>seed</i> (rata-rata $\pm$ SD)	0,750 $\pm$ 0,002	0,745 $\pm$ 0,016	0,800 $\pm$ 0,002
N	<i>Ensemble</i> 3 <i>seed</i> (rata-rata tabel)	0,753	0,722	0,803
O	I + bigram <i>hashing</i> 2 ²¹	0,738	0,722	0,804
P	I + bigram <i>hashing</i> 2 ²⁰	0,742	0,716	0,803
Q1	I dengan $\tau = 4$	0,724	0,704	0,792
Q2	I dengan $\tau = 0,5$	0,747	0,743	0,807
R1	I + CE label GPT-4o	0,736	0,720	0,797
R2	I + bobot keyakinan <i>teacher</i>	0,755	0,735	0,799
S1	I + <i>pooling</i> berbobot idf ^{0,5}	0,748	0,737	0,801
S2	I + bobot token dipelajari	0,742	0,715	0,801
T	I dengan $\tau = 1$, 3 <i>seed</i>	0,755 $\pm$ 0,002	0,748 $\pm$ 0,015	0,808 $\pm$ 0,000
U	T, <i>ensemble</i> 3 <i>seed</i> (v2)	0,761	0,745	0,811
V	U + tokenizer dipangkas (min. 10)	0,754	0,748	0,801

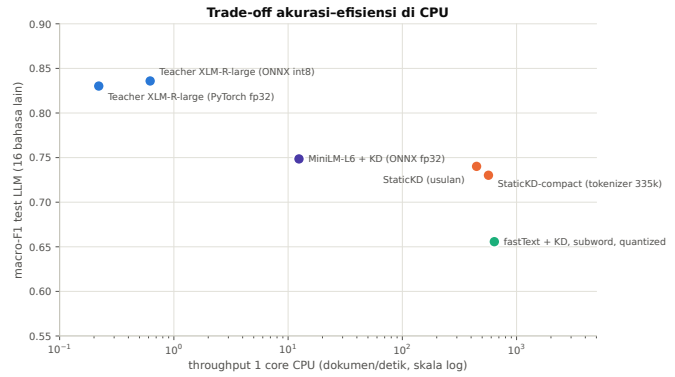
Perubahan sinyal distilasi atau *pooling* lainnya (R, S), kalibrasi bias per kelas (5-fold CV di dev: 0,721), dan pemotongan input ke 512 token (LLM16: 0,706) juga tidak membantu.

- **Temperatur adalah satu-satunya faktor yang konsisten.** Menurunkan τ dari 2 ke 1 (T vs M) menaikkan dev +0,005 (uji Welch tiga *seed*, $p = 0,014$) dan kesepakatan +0,008 ($p = 0,007$), tanpa perubahan di LLM16 ($p = 0,82$). Nilai $\tau = 4$ dan $\tau = 0,5$ lebih buruk.
- **StaticKD v2 (U)** menggabungkan $\tau = 1$ dengan *ensemble* tiga *seed*. Karena model linear terhadap tabelnya, rata-rata logit tiga model sama dengan logit satu model dengan tabel rata-rata, sehingga *ensemble* tidak menambah biaya inferensi. Hasilnya adalah dev 0,761 dan kesepakatan 0,811 (+0,011 terhadap I; $p < 0,001$), dengan ukuran, RAM, dan latensi yang identik dengan I. Versi tokenizer ringkasnya (V) setara dengan model I penuh.

[TODO: Tabel utama, uji signifikansi, analisis per bahasa, kalibrasi, dan *cascade* akan diperbarui ke StaticKD v2 dan dievaluasi pada *test set* manusia.]

E. Efisiensi (RQ4)

Tabel VII dan Gbr. 4–5 menunjukkan efisiensi seluruh model. StaticKD 720 kali lebih cepat dari *teacher* ONNX int8 dan 2.030 kali lebih cepat dari *teacher* PyTorch (*throughput* 1 *core*), serta 36 kali lebih cepat dari MiniLM-L6. Satu juta artikel per hari hanya membutuhkan sekitar 0,03 *core*. Kecepatannya setara fastText, tetapi akurasi lintas bahasanya jauh



Gbr. 4. Trade-off akurasi (macro-F1 LLM16) terhadap *throughput* 1 *core* CPU (skala log).

lebih baik. Pada StaticKD, sekitar 490 MB dari 505 MB RAM adalah struktur tokenizer Unigram, sedangkan tabel modelnya hanya 19 MB. Pemangkasan tokenizer (varian *compact*) menurunkan RAM menjadi 281 MB dengan biaya $-0,010$ macro-F1 lintas bahasa.

F. Analisis per Bahasa (RQ4)

Kinerja per bahasa diukur sebagai kesepakatan StaticKD dengan *teacher* pada *held-out* untuk 59 bahasa dengan minimal 20 dokumen *held-out* (Tabel VIII, Gbr. 6). Rata-rata kesepakatan per sistem tulisan adalah Arab 0,867, Sirilik 0,830, Latin 0,790, CJK/Thai/Myanmar 0,775, Indik 0,770, dan lainnya 0,745.

Temuannya adalah sebagai berikut.

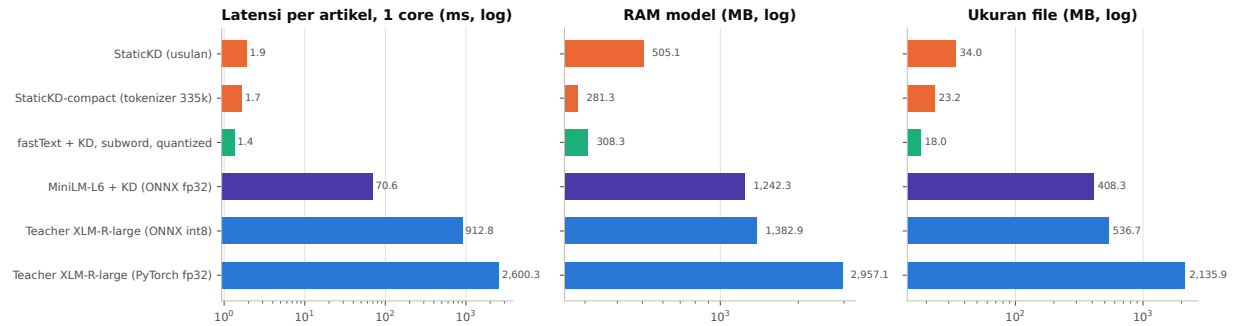
- 1) **Jumlah data dan keragaman sumber berkorelasi positif dengan kinerja.** Kelima bahasa terlemah (ps 0,22; ne 0,25; am 0,40; ms 0,47; ur 0,47) semuanya memiliki kurang dari 50 dokumen latih.
- 2) **Sistem tulisan tidak berpengaruh signifikan.** Bahasa beraksara non-Latin dengan data cukup berkinerja baik (ar 0,86; ru 0,87; hi 0,80; zh 0,82) berkat embedding yang selaras lintas bahasa. Sebaliknya, fastText + KD jatuh pada aksara non-Latin (be 0,06; pa 0,19; zh 0,41; ka 0,30), dan StaticKD lebih baik dari fastText di 94% bahasa.
- 3) **Keyakinan *teacher* adalah prediktor terkuat.** Pada bahasa tempat *teacher* sendiri ragu, *student* lebih sering berbeda, sehingga sebagian kelemahan adalah ketidakpastian *teacher* yang ikut tertiru.
- 4) **Anomali bahasa Turki.** Pada LLM16, akurasi bahasa Turki hanya 0,25 (*teacher* 0,65), padahal datanya cukup (sekitar 3.400 dokumen) dan kesepakatan *held-out*-nya 0,74. Dengan hanya 20 dokumen uji dan sumber berita yang homogen, kasus ini memerlukan investigasi lanjutan. Bahasa Indonesia berkinerja setara *teacher* (0,85 vs 0,85).

G. Analisis per Label

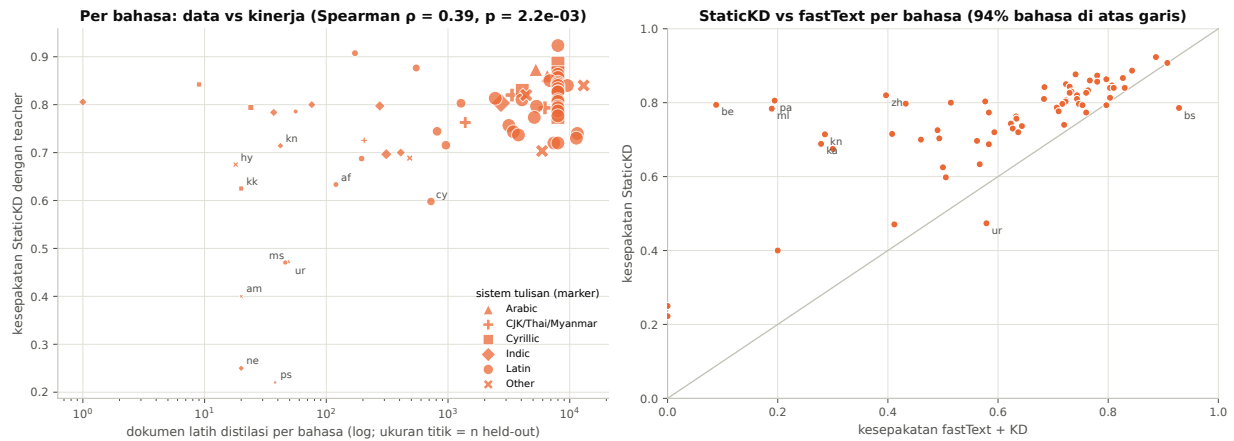
Label tersulit bagi StaticKD (F1 dev) adalah *lifestyle and leisure* (0,452), *human interest* (0,612), *society* (0,625), dan

Tabel VII
EFISIENSI CPU (P-Core TERPIN; 1C/4C = 1/4 Core)

Model	Latensi p50 / p95, 1c (ms)	Throughput 1c (dok/s)	Throughput 4c (dok/s)	RAM model (MB)	Ukuran file (MB)	Dependensi
Teacher PyTorch fp32	2.600 / 4.415	0,22	0,59	2.957	2.136	torch
Teacher ONNX int8	913 / 1.942	0,62	2,08	1.383	537	onnxruntime
MiniLM-L6 + KD	70,6 / 106,7	12,5	20,6	1.242	408	onnxruntime
StaticKD	1,94 / 6,31	447	1.683	505	34	numpy, tokenizers
StaticKD-compact	1,68 / 5,01	567	2.191	281	23	numpy, tokenizers
fastText + KD (q)	1,36 / 3,28	639	1.537	308	18	fasttext, tokenizers



Gbr. 5. Latensi per artikel, RAM model, dan ukuran file (skala log).



Gbr. 6. Kiri: kesepakatan per bahasa terhadap jumlah data latih (marker = sistem tulisan, ukuran = jumlah held-out). Kanan: StaticKD vs fastText per bahasa.

Tabel VIII
FAKTOR PENENTU KINERJA PER BAHASA

Uji	Statistik	p
Spearman: kesepakatan vs log(dokumen latih)	$\rho = 0,391$	0,002
Spearman: kesepakatan vs jumlah situs latih	$\rho = 0,349$	0,007
Spearman: kesepakatan vs keyakinan teacher	$\rho = 0,661$	<0,001
Kruskal-Wallis: antarsistem tulisan	$H = 8,69$	0,122
R^2 : log(data) / sistem tulisan / keduanya	0,109 / 0,129 / 0,213	—

science and technology (0,683), sedangkan label termudah adalah *sport* (0,940), *weather* (0,889), dan *health* (0,876) (Gbr. 7). Pola ini sama dengan teacher (*lifestyle* 0,667, *society* 0,697, *human interest* 0,740) dan dengan kesepakatan antaranotator manusia yang dilaporkan [1], tetapi lebih tajam. Selisih terbesar terhadap teacher ada pada *lifestyle* (−0,215) dan *science and technology* (−0,181), dua label yang sangat bergantung pada konteks, bukan kata kunci. Pada data dev

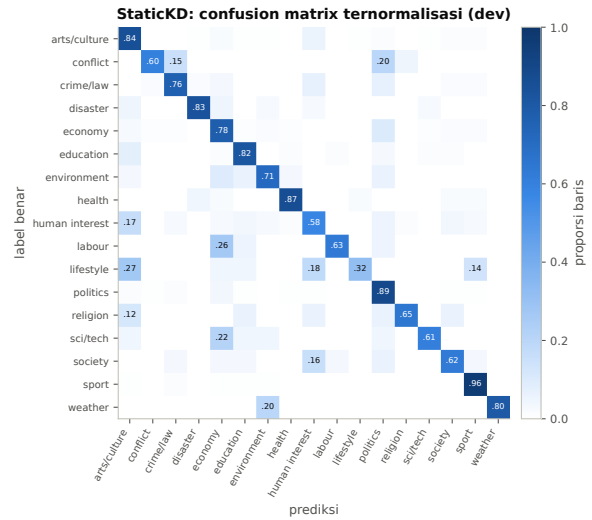
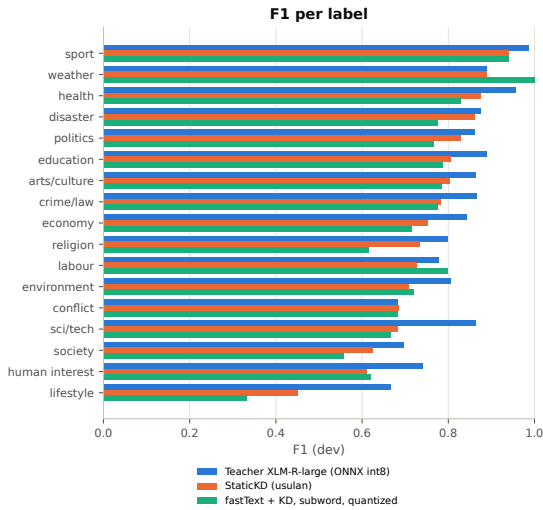
terdapat 117 dokumen (11,7%) yang benar di teacher tetapi salah di StaticKD, dan 50 dokumen sebaliknya.

H. Kalibrasi dan Mode Cascade

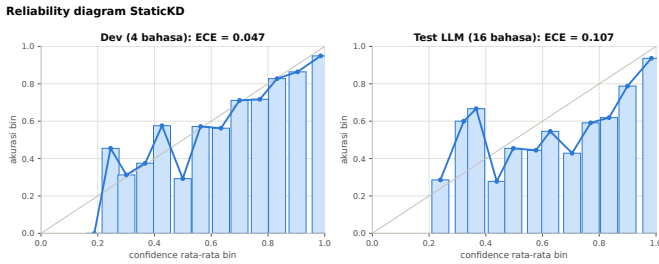
Probabilitas StaticKD terkalibrasi baik pada bahasa latih (ECE 0,047) dan cukup baik pada 16 bahasa lain (ECE 0,107, cenderung *overconfident*) (Gbr. 8). Hal ini memungkinkan mode *cascade*: artikel dengan *confidence* minimal θ diputuskan StaticKD, sedangkan sisanya dikirim ke teacher (Tabel IX, Gbr. 9). Dengan $\theta = 0,8$, StaticKD memutuskan dua pertiga artikel dengan akurasi 92,5%, dan *cascade* mencapai akurasi setara teacher dengan beban teacher tinggal sekitar 34%. *Throughput*-nya tetap dibatasi teacher, sehingga *cascade* cocok untuk jalur *batch*, bukan *real-time*.

V. PEMBAHASAN

Distilasi ke model static-embedding berhasil (RQ2). Model yang secara komputasi setara fastText mempertahankan



Gbr. 7. F1 per label (kiri) dan *confusion matrix* ternormalisasi StaticKD pada dev (kanan).



Gbr. 8. Diagram reliabilitas StaticKD pada dev dan LLM16.

Tabel IX

Cascade STATICKD KE *Teacher* ONNX INT8 (*Throughput* EFEKTIF 1 Core)

θ	Porsi StaticKD	Akurasi porsi	Micro-F1 dev	Macro-F1 LLM16	Throughput (dok/s)
0 (StaticKD saja)	100%	0,793	0,793	0,740	447
0,6	81,7%	0,871	0,840	0,804	3,4
0,7	72,9%	0,904	0,852	0,819	2,3
0,8	65,5%	0,925	0,854	0,845	1,8
<i>Teacher</i> saja	0%	—	0,860	0,836	0,62

sekitar 90% kinerja *teacher*, baik pada bahasa latih maupun pada 16 bahasa yang tidak pernah dilihat *teacher* saat *fine-tuning*. Dua faktor saling melengkapi. Pertama, *teacher* yang kuat lintas bahasa dapat melabeli data berita dalam bahasa apa pun tanpa biaya anotasi, sehingga keterbatasan data berlabel Kuzman (4 bahasa) dapat diatasi. Kedua, embedding statis yang selaras lintas bahasa memungkinkan pengetahuan dari bahasa kaya data berpindah ke bahasa lain. Faktor kedua inilah yang tidak dimiliki fastText.

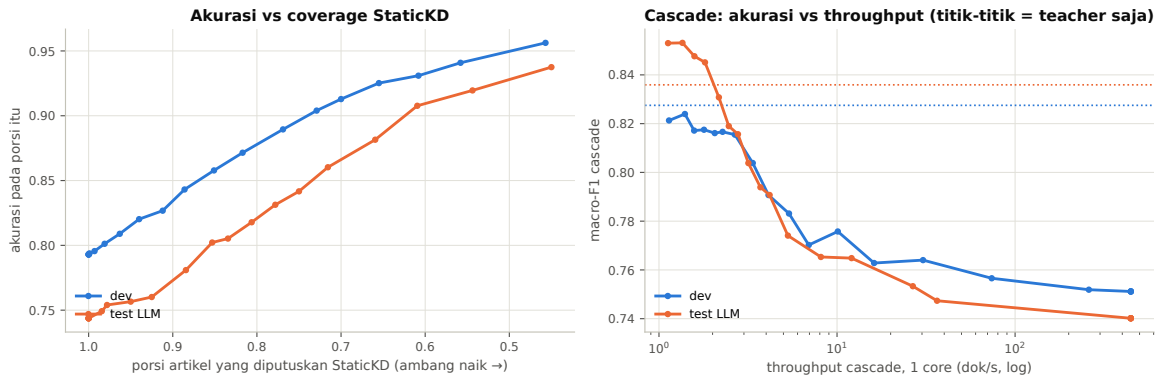
Mengapa model *bag-of-tokens* cukup? Topik berita sebagian besar ditentukan oleh kosakata (nama lembaga, istilah olahraga, istilah medis). Hal ini berbeda dari tugas yang sangat bergantung pada urutan dan konteks, seperti sentimen, sarkasme, atau inferensi. Konteks penuh transformer 6 *layer* hanya menambah 0,9–1,2 poin yang tidak signifikan dengan

biaya 36 kali lebih lambat. Batas kesalahan StaticKD ada pada label yang ambigu bahkan bagi manusia (*lifestyle*, *human interest*, *society*), di mana informasi konteks lebih berperan.

Batas kapasitas dan temperatur (RQ3). Seluruh upaya menambah kapasitas, yaitu data dua kali lipat, kepala MLP, bigram, bobot token, maupun *student* transformer dengan *attention*, tertahan di sekitar 0,75–0,76 macro-F1 dev. Satu-satunya perbaikan yang konsisten adalah temperatur yang lebih rendah. Temuan ini sejalan dengan literatur kesenjangan kapasitas [12], [13]: *student* berkapasitas sangat rendah tidak mampu merepresentasikan ekor distribusi *teacher*, sehingga target yang lebih tajam ($\tau = 1$) lebih bermanfaat daripada target yang lunak ($\tau \geq 2$). Kasus bigram juga menunjukkan bahwa kenaikan *fidelity* terhadap *teacher* dapat disertai penurunan akurasi terhadap label independen, sesuai pengamatan Stanton dkk. [14]. Karena itu *fidelity* tidak dapat dijadikan satu-satunya kriteria pemilihan *student*.

Posisi terhadap KD + LoRA. LoRA [5], [9] menurunkan biaya *training*, sedangkan masalah yang dihadapi adalah biaya inferensi. KD ke transformer kecil (setara KD-LoRA [6] tanpa adapter) memang mempercepat inferensi sekitar satu orde besaran (MiniLM: 12,5 dok/s), tetapi masih dua orde lebih lambat dari StaticKD dengan akurasi yang tidak berbeda signifikan. Selain itu, transformer kecil ini rentan terhadap kuantisasi int8, sedangkan StaticKD tidak memerlukan kuantisasi agresif karena biaya inferensinya sudah sangat rendah.

Implikasi praktis. Untuk *pipeline real-time* di server tanpa GPU, StaticKD dapat menggantikan *teacher* dengan biaya sekitar 0,03 *core* per satu juta artikel per hari, file 23–34 MB, dan hanya bergantung pada *numpy* dan *tokenizers*. Untuk bahasa target yang lemah, solusi termurah adalah menambah berita bahasa tersebut dan melabelinya dengan *teacher*, tanpa mengubah arsitektur. Bila dibutuhkan akurasi setara *teacher*, mode *cascade* memberikan titik tengah yang terkontrol melalui satu parameter θ .



Gbr. 9. Akurasi terhadap *coverage* StaticKD (kiri) dan akurasi *cascade* terhadap *throughput* (kanan; garis titik = *teacher* saja).

VI. KETERBATASAN

- 1) **Belum ada evaluasi pada *test set* manusia.** Data dev berlabel GPT-4o dan LLM16 berlabel LLM dengan satu anotator dan 20 dokumen per bahasa, sehingga interval kepercayaannya lebar. [TODO: Ganti butir ini setelah evaluasi pada *test set* manusia selesai.]
- 2) **Anotator LLM16 adalah Claude.** Anotasi dilakukan tanpa akses ke prediksi model dan memakai *prompt* Kuzman, dan kesepakatannya dengan *teacher* setara dengan kesepakatan antarmanusia. Meski demikian, validasi manusia pada sampel tetap diperlukan.
- 3) **Batas atas *student* adalah *teacher*.** Bias dan kesalahan *teacher* ikut tertiru, dan kesepakatan 70 bahasa mengukur *fidelity* terhadap *teacher*, bukan kebenaran absolut.
- 4) **Benchmark dilakukan pada satu perangkat** (laptop dengan CPU *hybrid* yang dipin ke P-core). Angka absolut di server akan berbeda, tetapi rasio antarmodel tetap informatif.
- 5) **Cakupan bahasa bergantung pada ketersediaan berita di CC-News.** Bahasa dengan data sangat sedikit (ps, ne, am, ur, ms) masih lemah.

VII. KESIMPULAN

Penelitian ini mengukur untuk pertama kalinya biaya CPU pengklasifikasi IPTC multibahasa Kuzman dan Ljubešić, yaitu 0,9–2,6 detik per artikel per *core*, sehingga tidak layak untuk *pipeline real-time* tanpa GPU. Kami mengusulkan StaticKD, distilasi *teacher* ke model *static-embedding* multibahasa yang dilatih dengan divergensi Kullback–Leibler pada 286 ribu artikel dari 70 bahasa, lalu dilipat menjadi tabel 17 logit per token. StaticKD mencapai macro-F1 0,751 (4 bahasa latih) dan 0,740 (16 bahasa lain), yaitu sekitar 90% kinerja *teacher*, dengan latensi 1,9 ms dan *throughput* 447 artikel per detik per *core*. Angka ini sekitar 720 kali lebih cepat dari *teacher* teroptimasi, dengan model 34 MB tanpa PyTorch. Ablasi menunjukkan bahwa kunci keberhasilannya adalah kombinasi distilasi pada data multibahasa dan inisialisasi embedding yang selaras lintas bahasa, dan bahwa *student bag-of-tokens* memiliki batas kapasitas yang hanya sedikit bergeser dengan temperatur $\tau = 1$ dan *ensemble* tabel (StaticKD v2, macro-F1 dev 0,761). Kelemahan per bahasa ditentukan oleh jumlah

data dan ketidakpastian *teacher*, bukan oleh sistem tulisan. Penelitian lanjutan mencakup evaluasi pada *test set* manusia multibahasa, penambahan data untuk bahasa lemah, perluasan ke label IPTC level-2 secara hierarkis, dan pemanfaatan probabilitas terkalibrasi untuk klasifikasi multi-label.

KETERSEDIAAN KODE DAN DATA

Seluruh kode (pengumpulan data, pelabelan *teacher*, *training*, evaluasi, *benchmark*, dan analisis) serta notebook Google Colab yang mereproduksi semua tabel dan gambar tersedia di [TODO: URL repositori publik]. EMMediaTopic tersedia di CLARIN.SI (<http://hdl.handle.net/11356/1991>), dan CC-News multibahasa tersedia di Hugging Face [21].

UCAPAN TERIMA KASIH

[TODO: Ucapan terima kasih (pembimbing, pendanaan, fasilitas).] [TODO: Pernyataan penggunaan AI sesuai kebijakan jurnal, misalnya: “Penulis menggunakan asisten AI (Claude, Anthropic) untuk membantu penulisan kode eksperimen, analisis, dan penyusunan draf naskah, serta sebagai anotator LLM16; seluruh hasil telah diperiksa dan menjadi tanggung jawab penulis.”]

REFERENSI

- [1] T. Kuzman and N. Ljubešić, “LLM teacher-student framework for text classification with no manually annotated data: A case study in IPTC news topic classification,” *IEEE Access*, vol. 13, pp. 35 621–35 633, 2025.
- [2] International Press Telecommunications Council (IPTC), “Media topics – IPTC NewsCodes,” [Online]. Available: <https://iptc.org/standards/media-topics/>, 2024.
- [3] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, “Unsupervised cross-lingual representation learning at scale,” in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2020, pp. 8440–8451.
- [4] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, “Bag of tricks for efficient text classification,” in *Proc. 15th Conf. European Chapter Assoc. Comput. Linguistics (EACL)*, 2017, pp. 427–431.
- [5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.
- [6] R. Azimi, R. Rishav, M. Teichmann, and S. Ebrahimi Kahou, “KD-LoRA: A hybrid approach to efficient fine-tuning with LoRA and knowledge distillation,” in *Proc. 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, ser. PMLR, vol. 262, 2024, pp. 73–80.

- [7] T. Kuzman, I. Mozetič, and N. Ljubešić, “Automatic genre identification for robust enrichment of massive text collections: Investigation of classification methods in the era of large language models,” *Mach. Learn. Knowl. Extraction*, vol. 5, no. 3, pp. 1149–1175, 2023.
- [8] T. Kuzman *et al.*, “State of the art in text classification for South Slavic languages: Fine-tuning or prompting?” arXiv:2511.07989, 2025.
- [9] O. Razuvayevskaya *et al.*, “Comparison between parameter-efficient techniques and full fine-tuning: A case study on multilingual news article classification,” *PLOS ONE*, vol. 19, no. 5, 2024.
- [10] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, “A survey on model compression for large language models,” *Trans. Assoc. Comput. Linguistics*, 2024.
- [11] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” NIPS Deep Learning and Representation Learning Workshop, arXiv:1503.02531, 2015.
- [12] J. H. Cho and B. Hariharan, “On the efficacy of knowledge distillation,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019.
- [13] S. I. Mirzadeh, M. Farajtabar, A. Li, N. Levine, A. Matsukawa, and H. Ghasemzadeh, “Improved knowledge distillation via teacher assistant,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, 2020.
- [14] S. Stanton, P. Izmailov, P. Kirichenko, A. A. Alemi, and A. G. Wilson, “Does knowledge distillation really work?” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.
- [15] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” arXiv:1910.01108, 2019.
- [16] W. Wang, H. Bao, S. Huang, L. Dong, and F. Wei, “MiniLMv2: Multi-head self-attention relation distillation for compressing pretrained transformers,” in *Findings Assoc. Comput. Linguistics: ACL-IJCNLP*, 2021, pp. 2140–2151.
- [17] S. Tulkens and T. van Dongen, “Model2Vec: Fast state-of-the-art static embeddings,” [Online]. Available: <https://github.com/MinishLab/model2vec>, 2024.
- [18] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” in *Findings Assoc. Comput. Linguistics: ACL*, 2024.
- [19] R. Piperno, L. Bacco, F. Dell’Orletta, M. Merone, and L. Pecchia, “Cross-lingual distillation for domain knowledge transfer with sentence transformers,” *Knowledge-Based Systems*, vol. 311, p. 113079, 2025.
- [20] M. Ulčar *et al.*, “Mono- and cross-lingual evaluation of representation language models on less-resourced languages,” *Computer Speech & Language*, 2025.
- [21] CloverSearch, “cc-news-multilingual: Common Crawl news split by language and year,” Hugging Face Datasets. [Online]. Available: <https://huggingface.co/datasets/CloverSearch/cc-news-multilingual>.
- [22] P. Koehn, “Statistical significance tests for machine translation evaluation,” in *Proc. Conf. Empirical Methods Natural Language Process. (EMNLP)*, 2004, pp. 388–395.
- [23] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [24] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.
- [25] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 4th ed. Thousand Oaks, CA, USA: SAGE, 2018.
- [26] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.